



Ache-Fuzz: Constraint-aware fuzzing for vulnerability discovery in distributed deep learning frameworks

Zhao Zhang ^a, Senlin Luo ^a, Liyuan Liu ^a, Limin Pan ^{a,*}

^a Information System & Security & Countermeasures Experiments Center, Beijing Institute of Technology, 100081, Beijing, PR China

ARTICLE INFO

Editor: Dr. A. Bertolino

Keywords:

Test samples
Constraint pattern
Hierarchical mutation
Deep learning library
Vulnerability mining

ABSTRACT

Ensuring the reliability and security of deep learning (DL) libraries is essential for the robustness of modern AI systems and large-scale intelligent computing infrastructures. However, the complexity of API semantics and the diversity of parameter constraints make it challenging to generate comprehensive and effective test cases. This paper presents Ache-Fuzz, a fuzzing-based automated testing framework designed to enhance vulnerability discovery in DL libraries such as TensorFlow. Ache-Fuzz integrates constraint-aware test generation with a hierarchical mutation strategy to construct diverse and valid API inputs. It extracts parameter constraint patterns from official API documentation to model structural and attribute dependencies, while the hierarchical mutation mechanism systematically strengthens boundary condition coverage and promotes broader exploration of API functionalities. Experimental evaluation on three versions of TensorFlow shows that Ache-Fuzz achieves over 25% API coverage and identifies 38 previously unknown vulnerabilities, 15 of which have been assigned CVE identifiers. These results demonstrate that Ache-Fuzz offers a scalable and effective approach for improving the robustness and security of large-scale AI software systems.

1. Introduction

With the rapid integration of deep learning (DL) into diverse domains such as image classification (Chen et al., 2022; Touvron, 2023; Li et al., 2023b; Yuan et al., 2020), video recommendation (Wang et al., 2023; Almeida et al., 2022; Tao et al., 2023; Wu et al., 2023), and autonomous driving (Deng et al., 2021; Cui, 2022; Chib and Singh, 2024; Badjie et al., 2025), ensuring the security and reliability of DL libraries has become a critical concern (Hatcher and Yu, 2018). Prominent libraries, including TensorFlow, Theano, and CNTK, serve as the foundational software infrastructure that abstracts the complexities of model design, training, and deployment through standardized Application Programming Interfaces (APIs). While these APIs significantly accelerate the development of AI applications, they also expand the potential attack surface. Similar to traditional software systems, DL libraries inevitably harbor latent vulnerabilities (Islam et al., 2019) that adversaries can exploit, potentially leading to denial-of-service (DoS) attacks, system crashes, or degraded performance in safety-critical environments. Real-world incidents, such as abnormal behaviors in autonomous driving systems from Tesla and Uber that contributed to safety failures, underscore the practical impact of such vulnerabilities. As DL libraries increasingly underpin large-scale AI-driven systems and critical infrastructure, robust and systematic vul-

nerability discovery is essential to ensuring their resilience and trustworthiness.

To uncover these hidden flaws, DL library testing has emerged as a fundamental technique for enhancing system-level security and robustness (Go et al., 2024). Current approaches can be broadly categorized into model-level testing (Zhang et al., 2025) and API-level testing (Ji et al., 2025). Model-level testing typically mutates pre-trained DL models and compares outputs across multiple backends to detect inconsistencies. Although this approach can reveal discrepancies caused by underlying library bugs, it suffers from two critical limitations: (1) distinguishing real bugs from benign inconsistencies (e.g., floating-point precision errors) requires carefully designed evaluation metrics, and (2) the scope of API coverage is often narrow and model-dependent. For instance, prominent frameworks like CRADLE (Pham et al., 2019) and LEMON (Islam et al., 2019) exercise only 35 and 59 APIs of TensorFlow, respectively. By contrast, API-level testing focuses on generating diverse and targeted test inputs for individual API functions, allowing for broader coverage and more direct identification of crashes and abnormal behaviors. Moreover, API-level testing does not rely on large datasets or complex orchestration logic, reducing the risk of accumulated precision losses and simplifying execution flow. These advantages make API-level testing a promising direction for scalable and effective

* Corresponding author.

E-mail address: panlimin2016@gmail.com (L. Pan).

vulnerability detection in DL libraries.

However, existing API-level DL library testing still has some limitations:

- Limited comprehensiveness of vulnerability discovery. Test cases derived from a small set of public API code examples typically cover only a subset of API functions, resulting in restricted functional coverage within DL libraries. The complexity of API input parameters contributes significantly to this challenge and can be analyzed from two perspectives. (1) Structural constraints: Many API input parameters require specific data structures, such as lists, tuples, or tensors, as defined in the official API documentation. Inputs that do not conform to the expected structure are rejected during preliminary validity checks. As a result, these invalid inputs are unable to exercise the core logic of the API functions. (2) Attribute constraints: Even when the correct data structure is provided, additional requirements must be satisfied regarding parameter attributes such as data type (dtype) and shape. For example, the dtype defines the expected type of the data, such as int32, float64, or string, while the shape specifies the dimensions of the data structure. A 4×5 matrix, for instance, represents a two-dimensional tensor with 4 elements in the first dimension and 5 in the second. Test inputs that violate these attribute constraints are similarly rejected by validity checks, preventing them from testing the deeper functionality of API implementations.
- Insufficient effectiveness of test cases. Existing mutation strategies often overlook important parameter attributes and boundary values, leading to reduced effectiveness in triggering vulnerabilities. For example, FreeFuzz (Wei et al., 2022,?) constructs valid input spaces for target APIs based on open-source code snippets, identifying the required types and shapes of input tensors, and then generates test cases through parameter mutation. However, due to the limited scope of the extracted parameter space and the rigidity of predefined mutation rules, the number of discovered vulnerabilities remains low. According to fuzzing theory, effective test inputs capable of triggering vulnerabilities are sparsely distributed within the entire input space. Most randomly generated inputs will not cause program failures, and only a small portion are likely to result in exceptions or crashes. Therefore, a key challenge is to improve the diversity and validity of test inputs in order to enhance the probability of discovering meaningful vulnerabilities while reducing the generation of invalid samples.

We propose a fuzzing-based testing method for DL libraries, Ache-Fuzz, which combines parameter constraint pattern extraction with a hierarchical mutation strategy. Ache-Fuzz constructs a unified constraint pattern to capture all relevant parameter constraints. This pattern is composed of seven key parameter attributes. For each attribute, representative phrases are extracted to match constraint rules specified in API documentation. Initial test inputs are then generated by fusing these attribute constraints to satisfy the expected input space of each API. Furthermore, Ache-Fuzz introduces a hierarchical mutation mechanism, in which parameter values are mutated at the attribute level. Each attribute is associated with an independent mutation ratio that is dynamically updated based on the outcomes of recent test executions.

The main contributions of this work are summarized as follows:

- We propose a high-coverage API-level vulnerability detection framework for DL libraries. The framework systematically generates valid test inputs that satisfy complex API input constraints, enabling vulnerability discovery across a large number of API functions.
- We introduce a dynamic hierarchical mutation mechanism that enhances the diversity of test cases by mutating parameter values at the attribute level. This approach adaptively adjusts mutation ratios based on recent test performance, leading to broader input diversity and an increased likelihood of triggering vulnerabilities within individual APIs.

- We conduct a comprehensive evaluation of Ache-Fuzz on multiple versions of TensorFlow. Ache-Fuzz successfully discovered 27 previously unknown vulnerabilities, of which 15 were assigned official CVE identifiers. These results demonstrate the effectiveness of the proposed method in uncovering real-world security issues in DL libraries.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 presents the Ache-Fuzz approach. Section 4 evaluates the approach on multiple versions of TensorFlow. Section 5 extends the approach to PyTorch. Section 6 discusses limitations and future work. Section 7 presents threats to validity, and Section 8 concludes the paper.

2. Related work

In this section, we reviewed prominent research on the vulnerability discovery in DL library. Based on technical characteristics, existing methods can be categorized into three types: model-based testing (Luo et al., 2021), API-based testing (Harzevili et al., n.d.; Chen et al., 2023) and large language model (LLM) based testing (Lyu et al., 2024).

2.1. Model-based testing

Pham et al. proposed CRADLE (Pham et al., 2019), a model-based testing framework that leverages two distance metrics to compare outputs across different backends for detecting inconsistencies. By tracing the propagation paths of anomalous outputs through the computational graph, CRADLE is able to localize the sources of discrepancies within DL systems. Wang et al. introduced LEMON (Islam et al., 2019), which enhances code coverage of DL libraries by automatically generating DL models through mutations at both the intact-layer and inner-layer levels. LEMON adopts a heuristic-guided strategy to steer model generation toward structures that are more likely to trigger inconsistencies, thereby increasing the likelihood of uncovering real bugs. AUDEE (Guo and et al., 2020) systematically constructs test inputs by varying model architectures, parameters, weights, and inputs, and employs three mutation strategies to enhance input diversity. For logical bugs, AUDEE utilizes heuristic-based cross-framework comparison (e.g., between TensorFlow and PyTorch) to detect behavioral inconsistencies. For NaN-related bugs, it generates models with output distributions that tend toward numerical abnormalities (e.g., excessively large or small values) to increase the likelihood of triggering such issues. Gu et al. presented Muffin (Gu et al., 2022), which uses differential testing across DL libraries to identify inconsistencies. Muffin generates diverse DL models by first constructing directed acyclic graphs for model architecture and then applying a greedy algorithm to determine layer-specific configurations. Building on the insight that two semantically equivalent graphs, even if they differ in APIs, data types, or optimization strategies, are expected to produce identical outputs, Wang et al. proposed EAGLE (Wan et al., 2022), a differential testing approach. EAGLE defines 16 graph equivalence rules to systematically construct pairs of equivalent models and identifies inconsistencies by comparing their outputs under identical input conditions.

2.2. API-based testing

Xie et al. proposed DocTer (Xie et al., 2022), a technique that extracts parameter constraints from API documentation using syntactic patterns derived from dependency parse trees. These constraints are then used to generate both valid and invalid inputs for testing DL library API functions. Zhang et al. introduced Predoo (Zhang and et al., 2021b), which extends numerical precision testing techniques to DL operators. It incorporates specialized mutation strategies and strictly adheres to operator shape constraints to generate valid test inputs, thereby facilitating precise error detection. To improve fuzzing effectiveness for DL

operators, Zhang et al. also proposed Duo (Zhang and et al., 2021a), a mutation-based fuzzing framework that employs a genetic algorithm to construct nine mutation operators for generating diverse tensor inputs. Duo performs differential testing across multiple operator instances to detect inconsistencies in their outputs. Deng et al. presented DeepREL (Deng et al., 2022), a framework that identifies similar API pairs by leveraging both syntactic and semantic information. It verifies the correctness of these pairs by analyzing invocation patterns and execution outputs. DeepREL then applies mutation-based fuzzing to generate diverse inputs, enabling the detection of behavioral inconsistencies between semantically related APIs. Wei et al. proposed FreeFuzz (Wei et al., 2022,?), a comprehensive API fuzzing framework that constructs input value spaces by dynamically collecting parameter types, values, and tensor shapes from invoked APIs. FreeFuzz employs three mutation strategies to generate test inputs, including type mutation, random value mutation, and database value mutation. It further utilizes both differential and metamorphic testing across different backends to identify potential vulnerabilities.

Shi et al. introduced ACETest (Shi et al., 2023), which automatically identifies input validation code within DL operators and extracts corresponding input constraints. These constraints are used to randomly generate test cases confined to valid input spaces, thereby enhancing the precision of fuzz testing. Li et al. proposed COMET (Li et al., 2023a), a framework designed to test DL layer APIs by generating diverse test inputs. It defines a set of mutation operators to produce varied layer inputs, parameter values, and layer sequences, while employing a coverage-guided search strategy to maximize code coverage across the DL framework. Liu et al. introduced Gandalf (Liu et al., 2023), which generates test inputs by assembling DL operators according to a context-free grammar tailored to specific task domains. Gandalf uses deep Q-networks to guide operator composition, aiming to maximize diversity in generated DL model structures and enhance the exploration of API behaviors. Deng et al. presented TENSORSCOPE (Deng et al., 2023), which extracts semantically equivalent APIs across different DL frameworks and formulates joint constraints to generate test cases. It further applies an error-guided fixing mechanism to iteratively refine test cases, improving their effectiveness in exposing inconsistencies and errors. Christou et al. proposed IvySyn (Christou et al., 2023), a type-aware fuzzing framework that statically analyzes local API type signatures to perform mutation-based fuzzing on low-level kernel implementations. Upon detecting memory safety violations, IvySyn automatically synthesizes human-readable code snippets in high-level languages to assist developers in debugging and vulnerability analysis. Yang et al. introduced ∇ Fuzz (Yang et al., 2023), which models DL library APIs as tensor or vector functions and applies differential testing to verify the correctness of automatic differentiation mechanisms. By comparing gradient outputs across implementations, ∇ Fuzz uncovers inconsistencies in backpropagation logic that could lead to incorrect training behavior.

2.3. LLM-based testing

Deng et al. introduced FuzzGPT (Deng et al., 2024), which is grounded in the hypothesis that code patterns responsible for historical errors may still contain components useful for triggering new faults. FuzzGPT leverages the generative and contextual understanding capabilities of large language models (LLMs), applying techniques such as fine-tuning and in-context learning to synthesize program inputs. These inputs are designed to maintain both syntactic correctness and semantic validity, improving the effectiveness of fuzz testing. In a follow-up work, Deng et al. proposed TitanFuzz (Deng et al., 2023), which utilizes LLMs to generate initial seed programs and further mutates them to produce new test cases. TitanFuzz incorporates prioritization strategies based on data dependency depth and the number of unique API calls in a test case. This allows it to construct test inputs with complex and diverse API interaction patterns, thereby enhancing the likelihood of uncover-

ing deep semantic bugs in DL libraries, particularly those arising from intricate API usage scenarios.

2.4. Other testing techniques

Besides fuzzing-based approaches, several mainstream testing techniques have been applied to DL libraries. Differential testing (Zhang et al., 2025) has been widely adopted to identify inconsistencies across multiple implementations of the same operator or model. For example, prior work compares outputs produced by TensorFlow, PyTorch, and MXNet to reveal numerical or semantic divergences. Metamorphic testing (Zhang et al., 2025) has also been frequently used to validate DL libraries by checking whether API outputs satisfy domain-specific metamorphic relations, such as input scaling, rotation, or invariance properties. These techniques are effective for detecting functional inconsistencies or semantic violations, but they typically require either multiple independent implementations or manually defined metamorphic relations. In contrast, Ache-Fuzz focuses on automatic vulnerability discovery at the API level, where security issues such as memory corruption and CHECK failures are often triggered by invalid input constraints rather than functional inconsistencies. Thus, Ache-Fuzz complements differential and metamorphic testing by targeting a different class of defects in DL frameworks.

2.5. Summary

In summary, existing approaches for testing DL frameworks, such as model-based, API-based, and LLM-based methods, have made significant contributions to improving test automation and increasing code coverage. However, these methods still face notable limitations in terms of comprehensiveness and effectiveness in discovering vulnerabilities.

Many API-level fuzzers rely heavily on a limited set of code snippets or static templates to generate test cases, which restricts their ability to cover the full range of APIs available in modern DL libraries. As outlined in prior work, structural constraints (e.g., required use of tensors, lists, or tuples) and attribute constraints (e.g., data type and shape) embedded in API input parameters are often overlooked or only partially considered. This results in a large number of invalid or shallow test cases that fail to reach the core logic of API implementations.

Furthermore, existing mutation strategies often lack the granularity needed to explore the parameter space effectively. They frequently ignore the semantic importance of individual parameter attributes or fail to adapt to feedback from execution results. Consequently, the generated test inputs tend to be either redundant or invalid, with a low probability of triggering meaningful exceptions or revealing vulnerabilities. Techniques like FreeFuzz demonstrate this limitation, as their predefined mutation rules and narrow value spaces struggle to expose deep-seated flaws in the framework.

Existing text mining and natural language processing (NLP) techniques have been used to extract structured information from unstructured documents, including API documentation. However, DL framework specifications typically contain highly formalized parameter definitions, shape constraints, and type requirements that can be captured reliably through rule-based parsing. In this context, a rule-based approach provides stronger determinism and higher precision than general-purpose NLP methods, which may introduce ambiguity or require supervised training data that is not available for this domain. Furthermore, documentation styles across DL libraries are relatively consistent, making rule-based extraction both feasible and effective. Nonetheless, exploring NLP-based extraction to improve generality and flexibility represents an interesting direction for future work.

These gaps highlight the need for a more principled approach that can systematically extract parameter constraints, generate diverse and valid test inputs, and dynamically guide mutations based on runtime feedback.

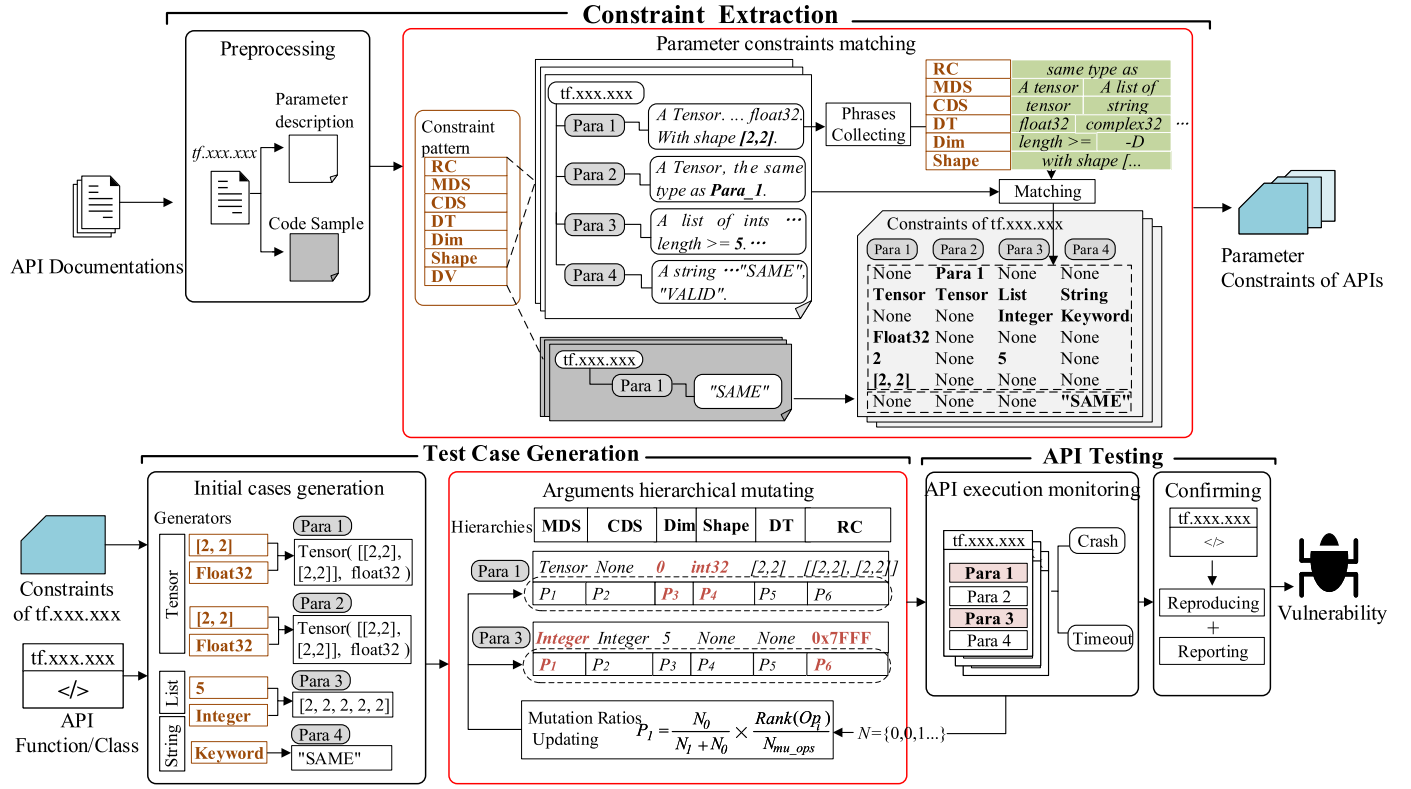


Fig. 1. The framework of Ache-Fuzz.

Args	Parameter description
gradients	A Tensor of type float32. Backpropagated gradients above the FakeQuantWithMinMaxVars operation.
inputs	A Tensor of type float32. Values passed as inputs to the FakeQuantWithMinMaxVars operation. min, max: Quantization interval, scalar floats.
min	A Tensor of type float32.
max	A Tensor of type float32.
num_bits	An optional int. Defaults to 8. The bitwidth of the quantization; between 2 and 8, inclusive.
narrow_range	An optional bool, Defaults to False. Whether to quantize into $2^{\text{num_bits}} - 1$ distinct values.
name	A name for the operation (optional)

Fig. 2. Parameter description.

3. Approach overview

In this section, we first introduce the overall architecture of DL library testing method Ache-Fuzz. Then, we discuss several important topics in Ache-Fuzz method.

3.1. Framework

The overall framework of Ache-Fuzz is illustrated in Fig. 1. It comprises three core modules: constraint extraction, test case generation, and API testing.

- The constraint extraction module takes API documentation as input and identifies the parameter constraints associated with each API, including structural and attribute-level rules.
- Based on these extracted constraints, the test case generation module produces valid and diverse test inputs tailored to the input specifications of each API.

```
tf.raw_ops.FakeQuantWithMinMaxVarsGradient(
    gradients, inputs, min, max, num_bits=8,
    narrow_range=False, name=None
)
```

Fig. 3. Code sample.

- The API testing module executes these test cases and observes the runtime behavior of the APIs. If any crashes, timeouts, or abnormal terminations occur, they are captured and further analyzed to determine whether they indicate potential vulnerabilities.

Although our implementation targets TensorFlow, the design of Ache-Fuzz is framework-agnostic because its constraint extraction and test generation pipeline rely solely on API documentation rather than framework-specific execution semantics.

3.2. Constraints extraction

3.2.1. Preprocess

The constraint extraction module is responsible for identifying parameter constraint information from API documentation by analyzing both textual descriptions and code examples. Parameter descriptions typically contain explicit constraint rules, including not only the number and names of parameters, but also detailed specifications of their expected data structures, value types, dimensional requirements, and other input constraints. For example, in the documentation of TensorFlow, each parameter is accompanied by a precise explanation of its semantics and associated constraints. As shown in Fig. 2, the parameter `gradients` is required to have a data structure of type `Tensor` and a value type of `float32`. In addition to textual descriptions, code examples often include default parameter values that implicitly reflect valid configurations. As illustrated in Fig. 3, the parameters `num_bit` and `narrow_range` are assigned default values of 8 and `False`, respectively. These default values comply with the input constraints and can be directly used to generate valid initial test cases. While a small number of APIs may lack formal parameter descriptions, such cases are rare and constitute only a minor portion of the overall API set.

Ache-Fuzz operates at the Python API layer of TensorFlow, where each Python operator invocation ultimately triggers the corresponding C++ kernel implementation. Although Ache-Fuzz does not directly instrument or analyze the underlying C++ kernels, the generated Python-level test cases are able to expose vulnerabilities within those kernels by supplying malformed or constraint-violating inputs. As a result, Ache-Fuzz targets the Python-facing attack surface of TensorFlow while indirectly exercising the C++ execution paths reached through these APIs.

3.2.2. Parameter constraints matching

To improve the representation of complex relationships among API parameter constraints, Ache-Fuzz introduces a structured constraint pattern. Each API in a DL library typically includes multiple parameters with distinct data structures and associated constraint rules, and dependencies may exist between these parameters. Manually generating parameter values based on such constraints is time-consuming and error-prone. To automate this process, Ache-Fuzz classifies parameter constraints into seven categories, which are used to guide test input generation. These categories are defined as follows:

- **Main Data Structure (MDS):** This denotes the primary data structure required by the parameter. In Python-based DL libraries, parameters may adopt standard types such as `Integer`, `List`, and `Tuple`, as well as specialized structures like `Tensor` and `DataType`, which are essential for neural network computations.
- **Child Data Structure (CDS):** This describes the data structure of the elements contained within a parameter. For composite structures such as `Tensor`, `List`, and `Tuple`, their elements are often structured as `Tensor` or `DataType`. For instance, if an API documentation describes a parameter as "a list of tensors", the main structure is `List`, and the child structure is `Tensor`. Ache-Fuzz uses this hierarchical representation to generate structurally valid parameter values.
- **Dimension (Dim):** This refers to the number of axes (or rank) of the data structure. DL parameters such as tensors or nested lists have dimensional constraints. For example, a 0-dimensional tensor (scalar) represents a single numeric or Boolean value; a 1D tensor (vector) corresponds to an array; a 2D tensor (matrix) contains rows and columns. Constraints on dimension are often specified in the documentation using expressions such as `N-D` or `length >= 1`. Ignoring these constraints may cause input validation to fail, preventing successful API invocation.
- **Shape:** This defines the length of the data in each dimension. For example, a parameter with a 2D tensor shape `[3, 4]` represents a matrix with 3 rows and 4 columns. Shape constraints are commonly ex-

Table 1

Rules and key phrases for each constraint type.

Types	Rules	Phrases
MDS	tensor	a tensor, -d tensor, ..., tensor
	integer	an int, scalar tensor, ..., int
	string	optional name, " or ", ..., str
	boolean	a bool, when true, ..., bool
	tuple	a tuple, shape of, ..., tuple
	dtype	tf.dtype, data type, ..., dtype
	list	a list, list of, array of, ..., list
CDS	float	a float value, ..., float
	tensorshape	a tensorshape, tensorshape
	tensor	tensor object, tensor
	integer	type int, int
	resource	type resource, resource
	float	type float, float
	dtype	tf.dtype, ..., dtype
Dim	X	length >= X, ..., X-D
	Shape	[X, ..., X]
	DT	List
RC	\	uint8, qint32, half, int8, int32, int64, float32, ..., complex128
	\	the same type, same type as the same length, same length as

pressed as with `shape [x, x]`, where `x` denotes the length of each dimension. Shape inherently implies dimension information; for example, `[3, 4]` indicates a tensor of rank 2.

- **Data Type (DT):** This constraint specifies the expected numeric type of a parameter, such as `int32`, `float64`, or `bfloat16`. If the input does not conform to the expected data type, the API will raise a type error. For example, `int32` refers to signed 32-bit integers. TensorFlow supports nearly 30 numeric data types.
- **Relationship Constraint (RC):** This constraint defines relationships between parameters. Two common forms include: (1) Same length: The current parameter must have the same length or dimension as another parameter (e.g., "must be the same length as X"). (2) Same type: The current parameter must share the same data type as another parameter (e.g., "must be the same type as X"). These inter-parameter constraints are crucial for maintaining consistency in multi-parameter APIs.
- **Default Value:** This refers to a predefined value either explicitly stated in the documentation or implied in example code. Since default values inherently satisfy all relevant constraints, they are treated as having the highest priority during test case generation and can be directly used to initialize valid inputs.

Ache-Fuzz performs constraint rule extraction by identifying and parsing key phrases associated with each constraint type from API documentation. The corresponding rules and representative key phrases for each constraint category are summarized in Table 1.

To ensure accurate rule identification, Ache-Fuzz adopts a priority-based keyword matching strategy, which assigns different matching priorities based on the structure and semantics of the key phrases:

- Single-word keyword phrases are assigned the lowest matching priority. For example, consider the phrase "a list of tensor" in an API description. If the keyword `tensor` is matched first, it may be incorrectly classified under the MDS constraint. However, a more accurate interpretation is that `list` belongs to MDS, while `tensor` belongs to the CDS constraint. By prioritizing more specific phrases such as "a list of" or "list of", the system can more precisely assign constraint types based on pre-defined feature-value mappings.
- For data type constraints, any recognized numeric type (e.g., `int32`, `float64`) is directly recorded and matched against the DT constraint category.
- For dimensional, shape, and relationship constraints, the system extracts surrounding contextual information to enable accurate

Table 2
Attributes for generating different data structures.

Object	Dim	Shape	CDS	DT	Default Value
Tensor	✓	✓	✓	✓	✓
List	✓	✓	✓	×	✓
Tuple	✓	✓	✓	×	✓
TensorShape	✓	✓	×	×	✓
String	×	×	✓	×	✓
Data type	×	×	×	×	✓
Integer	×	×	×	×	✓
Float	×	×	×	×	✓
Bool	×	×	×	×	✓

classification. For instance, when encountering a phrase such as "with shape [2, 2]", the expression is parsed to identify a shape constraint of [2, 2] and an implicit dimensionality constraint of 2.

- For inter-parameter relationship constraints, matching is based on detecting statements that imply shared properties, such as length or type, between the current parameter and another parameter. In such cases, the same constraint rules are applied to both parameters.
- Default value constraints are not associated with keyword phrases in Table 1, as default values are explicitly provided in documentation or code examples and do not require extraction via rule-based parsing. These values are directly usable as valid test inputs.

To illustrate the workflow of Ache-Fuzz, we use `tf.raw_ops.DrawBoundingBoxes` as a running example. A complete vulnerability instance triggered by this API is presented later in Section 4.7.2. According to its official documentation, the parameter *images* must be a 4-D Tensor of type `float32` or `half` with shape `[batch, height, width, depth]`, while *boxes* must be a 3-D Tensor of type `float32` with shape `[batch, num_boxes, 4]`. Ache-Fuzz converts these textual descriptions into the unified constraint pattern: for *images*, (MDS = Tensor, Dim = 4, Shape = `[b, h, w, d]`, DT ∈ {`float32`, `half`}); for *boxes*, (MDS = Tensor, Dim = 3, Shape = `[b, n, 4]`, DT = `float32`), together with a relationship constraint requiring the batch dimension of *boxes* to match that of *images*. These extracted constraints guide the subsequent synthesis and mutation of test inputs.

3.3. Test case generation

3.3.1. Initial test case generation

This module is responsible for generating initial test cases for a given API by leveraging the previously extracted parameter constraint rules. A set of parameter value generators is employed to handle various data structure types, ensuring that each parameter is initialized appropriately. For example, in TensorFlow, tensors can be instantiated using the function `tf.constant()` to conform to the required Tensor data structure.

The generation process prioritizes the use of predefined default values specified in API documentation or example code, as these values are guaranteed to satisfy the corresponding constraints. For parameters without default values, the module synthesizes test values based on the specific constraint types associated with each parameter. Table 2 summarizes the required constraint types used for generating values for different data structures.

Initially, if a parameter lacks explicit constraint rules, the module generates random values within reasonable bounds. For instance, if a parameter is defined as a list, the system first checks for applicable constraints on dimension, shape, and CDS. If such constraints exist, they guide the generation process; otherwise, random values are selected.

To enhance the quality and efficiency of the initial test suite, thereby accelerating the convergence of the fuzzing process, value ranges are defined for commonly used types. For example, integers and floating-point values are randomly selected within the range [0,10], while Boolean parameters are randomly assigned either `True` or `False`. These ranges help

ensure that test inputs are valid while maintaining diversity across samples.

Based on these extracted constraints, Ache-Fuzz generates an initial valid test case that satisfies the required data structures, shapes, and types. For example, the framework constructs a 4-D *images* tensor and a 3-D *boxes* tensor with a matching batch dimension: `arg 0 = tf.constant(np.random.random((1, 3, 2, 3)), dtype=tf.half)`, `arg 1 = tf.constant(np.random.random((1, 2, 4)), dtype=tf.float32)`, `arg 2 = ""`. This synthesized input respects all constraints of the API and ensures that `DrawBoundingBoxes` can be successfully invoked before mutation begins.

3.3.2. Arguments hierarchical mutation

In order to explore the API input space more thoroughly, Ache-Fuzz employs a hierarchical mutation algorithm that systematically modifies parameter values at multiple levels. A set of mutation operators is constructed to alter different hierarchical characteristics of each parameter, allowing the framework to generate diverse and boundary-revealing inputs. Furthermore, the mutation rates are dynamically adjusted according to the execution outcomes of previously generated test cases, which reduces the production of ineffective inputs and improves the efficiency of vulnerability discovery. This module takes the initial test case of an API as input and produces a set of mutated test cases as output.

Unlike the initial test cases, which are constructed directly from documentation-derived constraints, the mutated test cases are generated by systematically modifying and extending the initial inputs. Ache-Fuzz applies six mutation operators that target different hierarchical characteristics of each parameter value. These characteristics include the main data structure, the sub-data structure, the dimensionality, the shape, the numerical type, and the element values. Mutating parameters at multiple hierarchical levels enables Ache-Fuzz to exercise a broader space of boundary conditions in DL framework kernels. The six mutation operators are described as follows.

- Main data structure mutation. Ache-Fuzz randomly replaces the original data structure with another valid data structure (for example, replacing a tensor with a list).
- Sub-data structure mutation. Ache-Fuzz selects a sub-data structure within a parameter (for example, an inner list or nested tensor component) and replaces it with an alternative structure of the same category.
- Dimension mutation. Ache-Fuzz assigns a random integer to replace the original dimensionality, with values drawn from the range [-10, 10]. Negative and zero values are included because they frequently trigger crashes or assertion failures in DL libraries. Larger values are omitted due to excessive computational overhead.
- Shape mutation. Ache-Fuzz replaces the original shape with a randomly generated list of integers in the range [0, 10]. This mutation perturbs the size of each dimension and frequently exposes boundary-condition bugs.
- Numerical type mutation. Ache-Fuzz randomly selects an alternative numerical type from all types supported by the framework and replaces the original one.
- Element mutation. Ache-Fuzz mutates the values of individual elements by sampling random values in the full range allowed by the corresponding data type. This mutation applies to tensors, lists, integers, floating-point numbers, and Boolean parameters.

To ensure both the effectiveness and diversity of generated test cases, the constraint extraction module dynamically adjusts the feature mutation rates based on the runtime status of API calls. The feature mutation rate refers to the probability of altering a specific parameter attribute (or constraint feature) during test input generation. To thoroughly explore the boundary conditions of DL library behavior, these rates are adapted according to observed execution outcomes.

Specifically, if the number of successful API invocations decreases over time, the system interprets this as an indicator of excessive

mutation intensity and correspondingly reduces the mutation rate. Conversely, if the number of successful calls increases, the mutation rate is elevated to further explore the input space. This feedback-driven adaptation enables Ache-Fuzz to balance input diversity with input validity during fuzzing.

The execution status of API calls is inferred from the return codes of the underlying program. Since most DL libraries are implemented in languages such as Python and C/C++, the return code can be used to determine whether a test case executed successfully or triggered an exception. Typical return codes are interpreted as follows:

- Exit code 0: Indicates successful execution. The API is considered to have been called correctly with valid parameters.
- Exit code 1: Indicates a controlled failure, typically resulting from the API internal error handling mechanism rejecting invalid inputs. These cases are not considered vulnerabilities, but rather expected behavior in response to malformed test inputs.
- Negative exit Codes (e.g., -1073730791 or 0xC0000409): Suggest that the program accessed illegal memory or crashed unexpectedly. Such return codes indicate that the test case may have triggered a potential vulnerability in the DL library and warrant further investigation.

The hierarchical mutation process is described in Algorithm 1. Initially, a fixed-size sliding window of recent test execution outcomes is maintained, where each outcome is represented by its exit code. Based on the proportion of exit codes 0 (successful execution) and 1 (input rejection), the overall mutation intensity is dynamically adjusted. Specifically, if the number of 1-type outcomes exceeds that of 0, it suggests that a high proportion of test cases are invalid, prompting the system to reduce the mutation rate. Conversely, if successful executions dominate, the mutation rate is increased to promote input diversity and explore a wider space of API behaviors.

Furthermore, DL APIs often impose different levels of strictness on various parameter features. Mutations applied to certain features may more easily violate constraints and result in invalid test cases. For example, if the main data structure of a parameter is specified as Tensor, providing a value of an incompatible type (e.g., List or Integer) will trigger an `InvalidArgumentError`. This reflects the fact that main data structure constraints are generally more fundamental and rigid compared to constraints on child structures or data types.

The global mutation factor ρ can be calculated as follows:

$$\rho = \frac{N_{r0}}{N_r} \quad (1)$$

where ρ denotes the success ratio of executed test cases, N_r is the total number of test cases executed in the current window, N_{r0} is the number of test cases that returned exit codes 0 or 1 (i.e., valid executions).

Based on this, the mutation probability p_j for a mutation operator op_j is defined as:

$$p_j = \frac{\rho \times \text{rank}(op_i)}{m} \quad (2)$$

where m is the total number of mutation operators (set to 6 in this case), $\text{rank}(op_j) \in 1, 2, \dots, 6$ is the priority rank of operator op_j , corresponding to one of the six parameter features, p_j determines the likelihood of mutating the associated feature during test generation.

Ache-Fuzz then applies hierarchical mutation to diversify these valid inputs by perturbing attribute-level features such as shape, dimension, and data type. For instance, the mutation operators may enlarge the spatial dimensions of *images* or alter the structure of *boxes*, producing tensors whose shapes remain syntactically acceptable but lie near boundary conditions of the operator internal assumptions. This controlled expansion of the input space enables Ache-Fuzz to explore deeper execution paths and increases the likelihood of exposing robustness issues in the underlying kernel implementation.

Algorithm 1 Hierarchical mutation mechanism.

Input: Initial test case t_0 , number of test cases N_t , mutation operators set $Op = \{Op_i | 1 \leq i \leq 6, i \in Z^+\}$, initial mutation ratios set $P = \{P_j | 1 \leq j \leq 6, j \in Z^+\}$, number of return values N_r ;

Output: Generated test case set $T = \{t_n | 1 \leq n \leq N_t, n \in Z^+\}$;

```

1: Initialization: Return value set  $E = \{\}$ ,  $T = \{\}$ ,  $p_j = 0$ 
2: while  $\text{length}(T) < N_t$  do
3:   Randomly select parameter set  $Args$  from  $t_0$ 
4:   for each parameter  $arg_k \in Args$  do
5:     for each operator  $Op_i \in Op$  do
6:       Randomly generate floating-point  $q \in [0, 1]$ 
7:       if  $q < p_j$  then
8:         Using  $op_i$  to mutate  $arg_k$  for updating  $t_n$ 
9:       end if
10:    end for
11:  end for
12:  Add the updated  $t_n$  to  $T$ 
13:  Execute  $t_n$  to obtain the return value  $ret$ 
14:  if  $ret > 1 \vee ret < 0$  then
15:    break
16:  end if
17:  Add  $ret$  to  $E$ 
18:  if  $\text{length}(E) \geq N_r$  then
19:    Obtain the number  $N_{r1}$  of return value 1 from  $E$ 
20:  end if
21:  if  $N_{r1} \geq N_r$  then do
22:    break
23:  end if
24:  Obtain the number  $N_{r0}$  of value 0 from  $E$ 
25:  for  $p_j \in P$  do
26:    Obtain the ratio of return value using Eq.(1)
27:    Update mutation ratio using Eq.(2)
28:  end for
29: end while
30: return  $T$ 

```

3.4. API testing

3.4.1. API execution and monitoring

This module monitors the execution of each test case to determine whether a crash or timeout occurs during the API runtime. These two scenarios are considered potential indicators of vulnerabilities in the DL library. The definitions of crash and timeout used in this context are as follows:

- **Crash:** The API program terminates unexpectedly. The cause of the crash is inferred from the returned error type or exit code. For instance, a large negative exit code typically suggests a low-level kernel crash within the DL library. Exceptions such as `InvalidArgumentError` or `ValueError` indicate failures caused by invalid inputs. More severe indicators, such as `Check failed` or `Segmentation fault`, are treated as potential signs of exploitable vulnerabilities.
- **Timeout:** The program becomes unresponsive or exceeds a predefined execution time threshold. This may result from logic flaws or infinite loops triggered by malformed inputs. Timeout conditions are recorded for further analysis.

3.4.2. Vulnerability confirmation

To verify whether a crash or timeout is caused by an actual vulnerability, Ache-Fuzz re-executes the corresponding test case and performs a detailed analysis of the runtime logs. This process includes identifying which specific parameters triggered the exception and classifying the type of failure.

Table 3
Version information of TensorFlow.

Version	2.4.0	2.8.0	2.9.0
Lines of Python	260,869	262,503	266,703
Numbers of APIs	3945	4026	4129
APIs with descriptions	3353	3323	3446

Table 4
Experimental environment and configurations.

Component	Configuration
Operating System	Ubuntu 16.04.5 LTS
Anaconda Version	4.3.30
TensorFlow 2.4.0	Python 3.8, TensorFlow-GPU 2.4.0
TensorFlow 2.8.0	Python 3.9, TensorFlow-GPU 2.8.0
TensorFlow 2.9.0	Python 3.9, TensorFlow-GPU 2.9.0

Common indicators of critical vulnerabilities include messages such as `Check failed` and `Segmentation fault`, which suggest underlying issues in memory access or API robustness. However, not all timeouts indicate faults; in some cases, prolonged execution is part of the API intended behavior. For example, certain APIs are designed to wait for external input or perform network communication, naturally requiring more time.

Confirmed vulnerabilities are documented and summarized in structured reports, which are submitted to the responsible security team of the TensorFlow project for further review and mitigation.

When executing the mutated inputs, Ache-Fuzz monitors runtime behavior to detect CHECK failures or abnormal program termination. In this running example, calling: `tf.raw_ops.DrawBoundingBoxes(images=arg_0, boxes=arg_1, name=arg_2)` can reproduce the CHECK failure later assigned CVE-2022-36001, where the operator incorrectly handles certain shapes or types of the `boxes` argument. This demonstrates how Ache-Fuzz uncovers real-world vulnerabilities via constraint-guided test generation and mutation.

4. Experiments and analysis

4.1. Data source

The experimental dataset includes three versions of TensorFlow along with their corresponding API documentation. The details are summarized below:

- **TensorFlow Versions:** TensorFlow is one of the most widely adopted DL libraries. We deploy each version in an isolated Python environment to ensure consistency and avoid interference. Version 2.4.0, which is also used in FreeFuzz (Wei et al., 2022?), serves as a baseline for comparison. Versions 2.8.0 and 2.9.0 are newer releases used to evaluate the capability of Ache-Fuzz in discovering previously unknown vulnerabilities. The version information is summarized in Table 3.
- **API Documentation:** Ache-Fuzz collects API documentation from the official TensorFlow website to extract parameter constraint rules. A custom web crawler is developed to download and locally store the documentation for preprocessing. The number of APIs with available descriptions for each version is also listed in Table 3.

4.2. Experiment environment

Table 4 summarizes the experimental environment used for evaluating Ache-Fuzz. All experiments were conducted using TensorFlow versions 2.4.0, 2.8.0, and 2.9.0, each installed in a dedicated Python virtual environment managed through Anaconda3.

4.3. Evaluation metrics

To comprehensively evaluate the effectiveness of Ache-Fuzz, we adopt four metrics commonly used in both DL library testing:

- **APIs Successfully Invoked (ASI):** This metric counts the number of unique DL library APIs that are successfully invoked by generated test cases. An invocation is considered successful if the API executes without raising runtime errors in Python (e.g., `InvalidArgumentError`). Notably, invocations that trigger potential vulnerabilities are also counted as successful, as they indicate valid and executable inputs.
- **API Coverage Number (ACN) and API Coverage Rate (ACR):** These metrics represent the number of unique APIs covered (ACN) and the percentage of total available APIs covered (ACR) during testing. They are used to evaluate how extensively a testing method exercises the functionality provided by a DL framework. It is important to distinguish these metrics from the ASI as an API that is not directly invoked may still be indirectly triggered during the execution of another API. The API coverage rate is calculated as shown in Eq. (3):

$$ACR = \frac{N_{APIs_covered}}{N_{APIs_total}} \times 100\% \quad (3)$$

where $N_{APIs_covered}$ denotes the number of APIs actually covered during testing, and N_{APIs_total} represents the total number of available APIs in the corresponding version of TensorFlow.

- **Line Coverage Number (LCN) and Line Coverage Rate (LCR):** These metrics represent the total number of Python source lines covered during testing (LCN) and the percentage of covered lines relative to the total number of executable lines in the DL framework (LCR). Code coverage is a fundamental metric in software testing and is widely used to assess the thoroughness and effectiveness of test methods. The LCR is calculated as shown in Eq. (4):

$$LCR = \frac{N_{Lines_covered}}{N_{Lines_total}} \times 100\% \quad (4)$$

where $N_{Lines_covered}$ denotes the number of Python lines executed during testing, and N_{Lines_total} represents the total number of executable Python lines in a specific version of the framework.

- **Vulnerabilities Triggered (VT):** This metric counts the number of unique vulnerabilities identified by the testing process. The discovered vulnerabilities are categorized into three types:
 - *Fixed:* Vulnerabilities that have been resolved in subsequent TensorFlow releases.
 - *Previously Unknown:* Validated issues not documented or disclosed prior to this study.
 - *CVE-Assigned:* Subset of unknown vulnerabilities that have been officially assigned CVE identifiers.

4.4. Experimental parameters

After extracting parameter constraints from API documentation, Ache-Fuzz generates an initial test input for each API. The parameter N denotes the number of mutated test cases generated from each initial input using the hierarchical mutation mechanism. To evaluate the effectiveness of Ache-Fuzz under low sample generation cost, N is set to 100.

The parameter n represents the number of recent test executions considered when dynamically updating the mutation rates for the next test case. Instead of relying on the full execution history, Ache-Fuzz only considers the latest n results to adjust mutation probabilities, thus making the adaptation process more responsive. To balance responsiveness and statistical significance, n is set to 20.

Before any mutations are applied, the initial mutation probabilities of all operators, denoted as P_i , are set to 0. This ensures that the initial test inputs conform strictly to the extracted constraints, and mutation is only introduced progressively based on feedback during execution.

Table 5
Results of vulnerability discovery in comparative experiments.

Version	Method	ASI	ACN	ACR	LCN	LCR	VT
2.4.0	LEMON-0	62	62	1.57%	9942	3.81%	0
	LEMON	66	66	1.67%	10017	3.84%	0
	FreeFuzz	581	688	17.44%	8946	3.43%	15
	PCFuzz	581	688	17.44%	7559	2.90%	12
	Ache-Fuzz	675	1014	25.70%	9008	3.45%	20
2.8.0	LEMON-0	64	64	1.59%	7142	2.72%	0
	LEMON	67	67	1.66%	7193	2.74%	0
	FreeFuzz	383	401	9.96%	8796	3.35%	3
	PCFuzz	383	401	9.96%	7653	2.91%	3
	Ache-Fuzz	812	1032	25.63%	9227	3.52%	31
2.9.0	LEMON-0	63	63	1.53%	7181	2.69%	0
	LEMON	67	67	1.62%	7252	2.72%	0
	FreeFuzz	380	383	9.28%	8147	3.05%	3
	PCFuzz	380	383	9.28%	6704	2.51%	2
	Ache-Fuzz	708	1046	25.33%	8389	3.15%	30

4.5. Comparison experiment

To evaluate the effectiveness of Ache-Fuzz, we conduct a comparative experiment against several representative DL library testing methods. The comparison is performed on multiple versions of TensorFlow (TF), focusing on two primary metrics: API coverage and line coverage of the Python source code. These metrics reflect the breadth and depth of functional exploration within the DL library and provide insight into the ability of each approach to uncover potential vulnerabilities across a wider operational scope.

To ensure a fair and representative comparison, we selected four baseline methods that are closely related to Ache-Fuzz in terms of target and testing paradigm. LEMON-0 and LEMON are state-of-the-art DL testing frameworks that generate and mutate neural network models to exercise TensorFlow APIs, and thus serve as strong baselines for model-level testing. FreeFuzz is a documentation- and example-driven testing framework that targets TensorFlow APIs directly, making it a natural baseline for API-level fuzzing. PCFuzz is a Python coverage-guided fuzzer specifically designed for DL frameworks such as TensorFlow and PyTorch, which provides a representative comparison for coverage-guided input generation at the framework level. Together, these four methods cover both model-level and API-level testing, as well as random and coverage-guided fuzzing strategies, enabling a comprehensive evaluation of Ache-Fuzz.

- LEMON-0 (Pham et al., 2019): This is a simplified variant of the LEMON framework, where the heuristic model mutation strategy has been disabled. LEMON-0 executes test tasks on 12 pre-defined DL models. This configuration helps isolate the contribution of heuristic mutation to overall coverage, allowing us to assess its impact on API invocation and code line exploration.
- LEMON (Pham et al., 2019): LEMON employs a heuristic-based model mutation strategy to generate a large number of DL models for fuzz testing. Originally, it was designed to operate on TensorFlow 1.x versions. However, since API documentation for the 1.x series is no longer fully maintained, Ache-Fuzz cannot be directly compared in that setting. Therefore, to ensure fair comparison, we adapt LEMON to operate on TensorFlow 2.x. Additionally, both LEMON and LEMON-0 include a localization phase, which we remove in our adaptation due to its incompatibility with TensorFlow 2.x. This modification does not affect the evaluation of API and line coverage.
- FreeFuzz (Wei et al., 2022): FreeFuzz generates initial test cases by extracting code snippets from API documentation, developer-written test cases, and pretrained DL models. In its original implementation, test cases collected from TensorFlow 2.4.0 are reused across newer versions (2.8.0 and 2.9.0) for efficiency. Due to the high cost of rec-

collecting version-specific samples, the same approach is retained in our experiment.

- PCFuzz (Nie et al., 2024): PCFuzz is a Python coverage-guided fuzzing technique targeting DL frameworks such as TensorFlow and PyTorch. Unlike existing fuzzers that focus on high-level API testing with random inputs, PCFuzz leverages fine-grained Python line coverage to guide mutation and optimize input generation. By instrumenting the Python layer of DL frameworks with low overhead, PCFuzz significantly improves code coverage in low-level components and raw operations.

It is worth noting that we do not include more comprehensive automated testing frameworks, such as differential testing or metamorphic testing systems, as baselines. These techniques primarily target functional correctness by checking consistency across multiple DL frameworks or validating semantic properties under input transformations. In contrast, Ache-Fuzz focuses on discovering security and robustness issues rooted in low-level operator implementations. Since these frameworks have fundamentally different objectives and input models, a direct comparison would not yield meaningful conclusions. Therefore, our selected baselines reflect tools that are most closely aligned with Ache-Fuzz in terms of problem setting and testing granularity.

4.5.1. Experimental procedure

The comparison experiment evaluates each method by running its dynamic testing component once on each target version of TensorFlow. To collect API-level coverage metrics, we modify the *Hijack* tool from FreeFuzz (Wei et al., 2022), enabling the recording of both APIs invoked (i.e., successfully executed APIs) and APIs covered (including indirectly triggered APIs).

(1) For FreeFuzz, we follow its original setup by using the initial test cases it collects for TensorFlow 2.4.0. These samples are reused for versions 2.8.0 and 2.9.0 to ensure consistency and efficiency. FreeFuzz generates 1000 test cases per API during the mutation and fuzzing phases. Since some APIs may no longer exist in newer versions, we manually analyze the sample set to determine which APIs are still supported. This allows us to accurately report the number of APIs successfully invoked by FreeFuzz on each version.

(2) For LEMON-0 (Pham et al., 2019) and LEMON (Pham et al., 2019), test cases are generated at the model level rather than directly for APIs. Therefore, any API invoked during model execution is considered successfully invoked. We follow the original configuration of LEMON by generating 100 mutated models for each of the 12 initial models. To ensure fairness, LEMON-0 is also run 100 times for each initial model.

(3) For Ache-Fuzz, we select all APIs with available parameter descriptions (as identified in Section 4.4) as test targets. For each API, test cases are generated, executed, and monitored in real time. Metrics for API invocation, coverage, and code line coverage are collected during the testing process.

4.5.2. Results and analysis

Based on the experimental results presented in Table 5, we make the following observations:

(1) The proposed method achieves broader coverage of DL framework functionalities across all three versions of TensorFlow. Ache-Fuzz outperforms all baseline methods in terms of ASI, ACN, and ACR across TensorFlow versions 2.4.0, 2.8.0, and 2.9.0. These results demonstrate the ability of the method to test a wider range of framework APIs, thereby improving the coverage of vulnerability discovery. This performance is attributed to the parameter constraint pattern constructed by Ache-Fuzz, which accurately extracts constraint rules from API documentation. Leveraging these rules, the method generates valid parameter values that satisfy API input requirements, allowing more APIs to be successfully invoked and tested.

(2) The proposed method significantly improves the number of vulnerabilities discovered in DL frameworks. In terms of VT, Ache-Fuzz consistently outperforms the baseline approaches. Compared to FreeFuzz

(Wei et al., 2022), Ache-Fuzz discovers five additional vulnerabilities in TensorFlow 2.4.0 and over 30 additional vulnerabilities in both versions 2.8.0 and 2.9.0. Beyond broader API coverage, Ache-Fuzz applies dynamic, attribute-level mutation strategies across multiple parameter features with varying data structures, enabling the generation of diverse test inputs. This facilitates the thorough testing of parameter boundary conditions, ultimately leading to the discovery of more vulnerabilities.

(3) The proposed method demonstrates more stable performance across different versions of TensorFlow. Ache-Fuzz maintains consistent performance across all three TensorFlow versions, with ASI values exceeding 650, ACN exceeding 1000, ACR above 25%, and VT no fewer than 20. In contrast, baseline methods such as FreeFuzz suffer from reduced performance due to changes in the API set across versions; APIs may be deprecated or modified, rendering previously collected test cases unusable. Re-collecting valid examples manually imposes significant time costs. In contrast, Ache-Fuzz automatically generates version-specific test cases directly from API documentation, showing less sensitivity to framework updates.

(4) The proposed method offers a moderate improvement in code coverage. On TensorFlow versions 2.8.0 and 2.9.0, Ache-Fuzz achieves slight improvements in both LCN and LCR compared to baseline methods. While the gap among different methods in LCN and LCR is relatively small, the improvement reflects the ability of Ache-Fuzz to generate valid test cases for a larger number of APIs. However, compared to its substantial gains in ASI, ACN, ACR, and VT, the enhancement in code coverage is relatively limited. This is because many APIs internally reuse common components, leading to overlapping lines of code during execution.

4.6. Ablation experiment

The ablation experiment is conducted to prove whether the hierarchy mutation mechanism enhances the diversity of test cases. The number of vulnerabilities is used as the evaluation method. The final goal of the hierarchy mutation mechanism is to increase the possibility of triggering vulnerabilities in each API.

4.6.1. Experiment procedures

To evaluate the effectiveness of the proposed Arguments Hierarchical Mutation (AHM) module, we conducted an ablation study using the VT as the evaluation metric. Specifically, we constructed a baseline version of Ache-Fuzz (denoted as Base) by removing the AHM component responsible for hierarchical mutation of parameter values. We then performed vulnerability detection experiments on multiple versions of TensorFlow using both the baseline and the complete Ache-Fuzz approach.

To ensure fairness in test case generation, the baseline method, which does not include the mutation phase, was configured to generate 100 initial test cases per API. This number corresponds to the total number of test cases produced by Ache-Fuzz after mutation. Experimental results were collected and compared across different versions to evaluate the impact of the AHM module on the effectiveness of vulnerability discovery.

4.6.2. Results and analysis

The experimental results demonstrate that the AHM module in Ache-Fuzz significantly enhances the effectiveness of vulnerability discovery in DL frameworks. As shown in Table 6, the incorporation of the AHM module more than doubles the number of vulnerabilities identified compared to the baseline method across all three evaluated versions of TensorFlow.

The baseline method generates a large number of initial test cases based on parameter constraint rules extracted from API documentation. While these test cases contain parameter values that satisfy the input constraints and can successfully invoke APIs, they are often insufficient for thoroughly testing the boundary conditions of the framework, which are critical for exposing hidden defects.

Table 6

The results of ablation experiment.

Method	TF 2.4.0	TF 2.8.0	TF 2.9.0
Base	9	15	13
Base + AHM	20	31	30

By contrast, the AHM module applies a set of hierarchical mutation operators to diversify the parameter values within the initial test cases. This mutation process increases the heterogeneity of test inputs and enables broader exploration of the API input space, particularly at the boundary conditions. As a result, it substantially improves the likelihood of triggering abnormal behaviors during API execution, leading to the discovery of additional vulnerabilities within the DL framework.

4.7. Vulnerability mining experiment

4.7.1. Experiment procedures

In the vulnerability discovery experiment, all potential vulnerabilities identified by the proposed method were recorded and systematically validated across multiple versions of TensorFlow, including versions 2.4.0, 2.8.0, 2.9.0, and the latest available release. For each discovered issue, we determined whether it had already been addressed in subsequent versions, thereby classifying the vulnerabilities into fixed and unknown categories.

To verify the unknown vulnerabilities, we prepared detailed vulnerability reports for each unresolved issue and submitted them to the TensorFlow development team for confirmation. Based on their feedback, we further categorized the confirmed unknown vulnerabilities and recorded the number of those that were officially assigned CVE identifiers. This process ensured both the technical soundness of our findings and their practical relevance to the security of DL frameworks.

4.7.2. Results and analysis

The experimental results presented in Table 8 demonstrate that Ache-Fuzz is highly effective in uncovering vulnerabilities within DL frameworks. Furthermore, it exhibits strong practical value by successfully identifying previously undisclosed vulnerabilities.

Using Ache-Fuzz, a total of 38 vulnerabilities were discovered across three versions of TensorFlow. Among these, 11 have been fixed in the latest official release, while 27 were confirmed as previously unknown vulnerabilities. Notably, 15 of these newly identified vulnerabilities have been assigned official CVE identifiers, underscoring the practical impact and significance of the Ache-Fuzz.

To clarify the official status of the discovered issues, we reported all 38 findings to the TensorFlow Security Team following the responsible disclosure process. Among them, 15 vulnerabilities were confirmed as security issues and assigned CVE identifiers by either the TensorFlow team or MITRE (e.g., CVE-2022-36001). The remaining cases were acknowledged as stability or robustness bugs rather than security vulnerabilities due to their limited exploitability. For each confirmed CVE, the TensorFlow developers also provided a root-cause characterization (e.g., improper input validation, shape inconsistencies, or unchecked internal assumptions), and corresponding fixes were incorporated into subsequent releases. A summary of these CVEs is provided in Table 7. Beyond CVE status, we further categorize the confirmed vulnerabilities using the CWE classifications reported by NVD and the TensorFlow Security Team. As shown in Table 7, the vast majority fall under CWE-617 (Reachable Assertion), which is the dominant vulnerability pattern in TensorFlow operator kernels, while one case corresponds to CWE-476 (NULL Pointer Dereference). These categories represent the official MITRE-aligned descriptions of the confirmed security flaws. For the remaining non-CVE cases, no corresponding CWE category exists, as these issues arise from undocumented or implicitly assumed input constraints that lead to execution failures. They therefore represent previously

Table 7
CVE-assigned vulnerabilities and corresponding CWE classifications.

ID	CVE Number	API Name	CWE ID / Weakness Type
1	CVE-2022-35995	tf.raw_ops.AudioSummaryV2	CWE-617 / Reachable Assertion
2	CVE-2022-35994	tf.raw_ops.CollectiveGather	CWE-617 / Reachable Assertion
3	CVE-2022-35971	tf.raw_ops.FakeQuantWithMinMaxVars	CWE-617 / Reachable Assertion
4	CVE-2022-35993	tf.raw_ops.SetSize	CWE-617 / Reachable Assertion
5	CVE-2022-35952	tf.raw_ops.UnbatchGrad	CWE-617 / Reachable Assertion
6	CVE-2022-35935	tf.raw_ops.SobolSample	CWE-617 / Reachable Assertion
7	CVE-2022-35992	tf.raw_ops.TensorListFromTensor	CWE-617 / Reachable Assertion
8	CVE-2022-35991	tf.raw_ops.TensorListScatter	CWE-617 / Reachable Assertion
9	CVE-2022-36004	tf.random.gamma	CWE-617 / Reachable Assertion
10	CVE-2022-36003	tf.raw_ops.RandomPoissonV2	CWE-617 / Reachable Assertion
11	CVE-2022-36002	tf.raw_ops.Unbatch	CWE-617 / Reachable Assertion
12	CVE-2022-36001	tf.raw_ops.DrawBoundingBoxes	CWE-617 / Reachable Assertion
13	CVE-2022-36000	tf.raw_ops.Eig	CWE-476 / NULL Pointer Dereference
14	CVE-2022-36005	tf.raw_ops.FakeQuantWithMinMaxVarsGradient	CWE-617 / Reachable Assertion
15	CVE-2022-35990	tf.raw_ops.FakeQuantWithMinMaxVars	CWE-617 / Reachable Assertion
		PerChannelGradient	CWE-617 / Reachable Assertion

CVE-2022-36001

Impact

When 'DrawBoundingBoxes' receives an input 'boxes' that is not of dtype 'float', it gives a 'CHECK' fail that can trigger a denial of service attack.

```
import tensorflow as tf
import numpy as np
arg_0=tf.constant(value=np.random.random(size=(1, 3, 2, 3)), shape=(1, 3, 2, 3),
dtype=tf.half)
arg_1=tf.constant(value=np.random.random(size=(1, 2, 4)), shape=(1, 2, 4),
dtype=tf.float32)
arg_2=""
tf.raw_ops.DrawBoundingBoxes(images=arg_0, boxes=arg_1, name=arg_2)
```

(a) tf.raw_ops.DrawBoundingBoxes

CVE-2022-36004

Impact

When 'tf.random.gamma' receives large input shape and rates, it gives a 'CHECK' fail that can trigger a denial of service attack.

```
import tensorflow as tf
arg_0=tf.random.uniform(shape=(4,), dtype=tf.int32, maxval=65536)
arg_1=tf.random.uniform(shape=(4, 4), dtype=tf.float64, maxval=None)
arg_2=tf.random.uniform(shape=(4, 4, 4, 4), dtype=tf.float64, maxval=None)
arg_3=tf.float64
arg_4=48
arg_5='None'
tf.random.gamma(shape=arg_0, alpha=arg_1, beta=arg_2, dtype=arg_3,
seed=arg_4, name=arg_5)
```

(b) tf.random.gamma

Fig. 4. The samples of vulnerabilities.

Args		Args	
image	A Tensor. Must be one of the following types: float32, half. 4-D with shape [batch, height, width, depth]. A batch of images.	beta	A Tensor or Python value or N-D array of type dtype. Defaults to 1. beta provides the inverse scale parameter(s) of the gamma distribution(s) to sample. Must be broadcastable with alpha.

(a) 'image' in tf.raw_ops.DrawBoundingBox

(b) 'beta' in tf.random.gamma

Fig. 5. The description of parameters.

Table 8

Summary of discovered vulnerabilities in DL frameworks.

Vul type	Confirmed	Fixed	Discovered	CVEs
Number	38	11	27	15

unreported robustness problems rather than standardized security weakness types.

Fig. 4 provides representative examples of vulnerabilities uncovered by Ache-Fuzz. In Fig. 4a, the vulnerability resides in the tf.raw_ops.DrawBoundingBoxes API, which is triggered when the images parameter is assigned a data type of tf.half. According to the API specification (see Fig. 5a), this parameter supports both float32 and half; however, in certain input configurations, the use of tf.half leads to a CHECK failure, indicating a potential internal assertion failure within the TensorFlow kernel, which may result in a DoS condition. Fig. 4b illustrates another vulnerability found in the tf.random.gamma API. When the beta parameter is assigned an exces-

sively large and high-dimensional tensor (e.g., shape = [4, 4, 4, 4, 4]) with dtype=tf.float64, the API triggers a CHECK failure. According to the official documentation (see Fig. 5b), the beta parameter must be broadcastable with alpha, yet no explicit constraints are enforced at the documentation or runtime level. This inadequate validation allows malformed inputs to reach deeper parts of the execution pipeline, potentially causing crashes or resource exhaustion.

These examples demonstrate the capability of Ache-Fuzz to generate semantically valid yet stress-inducing test inputs that exploit subtle parameter constraints, such as data types, shapes, and broadcasting compatibility. Such vulnerabilities can be weaponized by adversaries to launch DoS attacks or induce unstable behavior in deployed DL systems.

5. Extension to PyTorch

To provide preliminary evidence of the generalizability of Ache-Fuzz beyond TensorFlow, we conducted a small-scale case study on PyTorch, as shown in Fig. 6. Specifically, we instantiated Ache-Fuzz on the torch.nn.Conv2d API, whose documentation specifies that the

```

import torch
import torch.nn as nn

conv = nn.Conv2d(in_channels=3, out_channels=16, kernel_size=3)

# Valid input based on extracted constraints
x = torch.randn(1, 3, 224, 224)

# Mutated input that violates the 4-D shape constraint
x_mut = x.view(1, -1)

# This call triggers a runtime error in PyTorch:
# RuntimeError: expected 3D (unbatched) or 4D (batched) input to
# conv2d, but got 2D input
conv(x_mut)

```

Fig. 6. A minimal Ache-Fuzz example applied to a PyTorch API (Conv2d).

input must be a 4-D tensor of shape $[N, C, H, W]$ (batch size, number of channels, height, width).¹ Following the same workflow as in TensorFlow, we first extracted parameter constraints from the API documentation (e.g., $MDS = \text{Tensor}$, $Dim = 4$, $Shape = [N, C, H, W]$, and consistency between the input channels and the `in_channels` attribute of the layer). Ache-Fuzz then generated a valid initial test case and applied hierarchical mutation at the attribute level. For example, starting from a valid input tensor of shape $(1, 3, 224, 224)$, a mutation that flattens the spatial dimensions into a 2-D tensor causes Conv2d to raise a runtime error (`RuntimeError: expected 3D (unbatched) or 4D (batched) input to conv2d, but got 2D input`). This case study shows that the constraint-aware generation and mutation strategy of Ache-Fuzz can be transferred to PyTorch APIs, although fully automating constraint extraction for PyTorch would require additional engineering effort due to its less uniform documentation structure.

6. Discussion

Ache-Fuzz demonstrates strong capability in discovering vulnerabilities caused by abnormal parameter values, particularly those that result in `Check fail` errors, which are among the most common types of vulnerabilities in TensorFlow. These failures are typically triggered by inputs that violate internal assumptions, such as zero-dimensional tensors, negative values in restricted contexts, or incompatible shapes. In addition, Ache-Fuzz is also effective at identifying `Segfault`-type vulnerabilities, which are often induced by memory access violations resulting from malformed or unexpected arguments. Both `Check fail` and `Segfault` errors can be exploited by attackers to launch DoS attacks against systems using TensorFlow, making these vulnerabilities highly relevant in practice.

One key advantage of Ache-Fuzz lies in its adaptability to different TensorFlow versions. As new versions are released and APIs are added, removed, or modified, Ache-Fuzz only requires updated API documentation to resume effective fuzzing. Because it extracts parameter constraints directly from these documents, which are updated alongside the APIs themselves, the approach remains both timely and maintainable across versions.

Furthermore, Ache-Fuzz exhibits strong generalizability. The core idea of leveraging structured parameter constraints and hierarchical mutation for vulnerability discovery is applicable to other DL libraries beyond TensorFlow. Although different frameworks may adopt varying documentation styles or syntax to express parameter constraints, adapting Ache-Fuzz only requires modifying the constraint extraction component. Given that most DL libraries expose their functionalities via APIs,

the methodology proposed in this work can serve as a practical and extensible template for testing other frameworks such as PyTorch or MXNet.

However, Ache-Fuzz also has certain limitations. It performs less effectively in detecting complex types of vulnerabilities, such as use-after-free errors, memory corruption, or issues caused by malicious payloads embedded within model files. The discovery of such vulnerabilities typically requires a comprehensive understanding of the internal structure of the framework, including both source code and compiled components written in multiple languages such as Python, C++, and CUDA. In addition, identifying these issues demands expertise in low-level memory management and control flow analysis, which falls outside the scope of API-level fuzzing. Addressing these challenges may require combining fuzzing with static analysis, symbolic execution, or manual auditing conducted by experienced security professionals.

Future work may extend Ache-Fuzz to additional DL libraries and programming languages, and explore the use of NLP techniques to further generalize and automate the constraint extraction process.

7. Threats to validity

Following the validity threat classification proposed by Feldt and Magazinius (Feldt and Magazinius, 2010), we discuss the potential threats to the validity of our study from the perspectives of construct validity, internal validity, conclusion validity, and external validity.

Construct validity. Construct validity concerns whether the employed techniques and measurements accurately capture the intended research constructs. Ache-Fuzz relies on structured and well-specified API documentation to extract parameter constraints such as data types, shapes, and default values. While TensorFlow provides relatively consistent and comprehensive operator documentation, the quality and formality of documentation vary across DL frameworks. Ambiguous, incomplete, or informal specifications may negatively affect the accuracy of constraint extraction and, consequently, the quality of generated test inputs. This limitation is inherent to documentation-driven testing approaches. Nevertheless, official API documentation remains the primary and authoritative specification used by developers and testers, making it a reasonable and widely adopted foundation for input modeling.

Internal validity. Internal validity addresses whether the observed experimental effects can be attributed to the proposed approach rather than unintended factors. A subset of TensorFlow APIs contains parameters without default values. For such parameters, Ache-Fuzz must resort to heuristic or randomized input synthesis when generating initial test cases, which may reduce their precision and affect early-stage fuzzing effectiveness. This limitation may introduce bias for APIs with complex parameter dependencies. While this does not undermine the overall applicability of Ache-Fuzz, APIs of this type may benefit from additional human-guided seed inputs or domain-specific specifications, which we leave as future work.

Conclusion validity. Conclusion validity concerns the extent to which the experimental results support the drawn conclusions. Our evaluation employs API coverage, line coverage, and the number of confirmed crashes as primary effectiveness indicators. Although these metrics are widely used in prior DL library testing research, they cannot fully capture all aspects of security impact or semantic correctness. In particular, not all observed crashes correspond to exploitable vulnerabilities. To mitigate this threat, we manually analyzed crash reports, reproduced failure cases, and validated vulnerabilities through confirmation and, when possible, official CVE assignment.

External validity. External validity considers the generalizability of the results beyond the evaluated settings. Ache-Fuzz currently targets the Python API surface of TensorFlow and does not directly instrument lower-level C++ kernels, graph execution paths, or TFLite execution environments. Although Python-level API calls eventually trigger underlying C++ operators and can indirectly expose kernel-level vulnerabilities, the approach does not aim to provide exhaustive coverage of the

¹ See the official PyTorch documentation for Conv2d.

entire TensorFlow attack surface. Furthermore, while a PyTorch case study demonstrates the transferability of the core ideas of Ache-Fuzz, fully automated support for additional frameworks requires non-trivial engineering effort due to differences in documentation structure, API conventions, and error-handling mechanisms.

TensorFlow is selected as the primary evaluation target because of its wide adoption, stable Python bindings, and well-structured documentation, which make it well suited for systematic constraint extraction and fuzzing. Nevertheless, focusing on a single framework and a limited set of baseline approaches naturally constrains the generalizability of our findings. Extending Ache-Fuzz to additional DL frameworks and execution environments remains an important direction for future work.

8. Conclusion

This paper presents Ache-Fuzz, a fuzzing framework designed to enhance the discovery of unknown vulnerabilities in DL frameworks by increasing both API coverage and the diversity of test cases. Ache-Fuzz introduces a unified constraint pattern based on seven key parameter attributes, each corresponding to a distinct category of constraint rules extracted from API documentation. Leveraging these rules, the framework generates valid initial test cases for each API. To further improve fuzzing effectiveness, Ache-Fuzz applies a hierarchical mutation strategy that perturbs parameter values across multiple attribute levels. The mutation probability for each level is dynamically adjusted according to the execution feedback from recent test cases, enabling the framework to explore input boundaries more effectively. Experimental evaluations conducted on three versions of TensorFlow demonstrate the effectiveness of Ache-Fuzz. The framework achieves over 25% API coverage and successfully uncovers 38 vulnerabilities, among which 11 have been confirmed as previously fixed, 27 are newly discovered, and 15 have been assigned official CVE identifiers. These results indicate that Ache-Fuzz significantly improves both the depth of API testing and the efficiency of vulnerability discovery in complex DL libraries. Future work will explore integrating Ache-Fuzz with additional analysis techniques to uncover deeper and more complex vulnerability types.

CRediT authorship contribution statement

Zhao Zhang: Writing – review & editing, Writing – original draft, Conceptualization; **Senlin Luo:** Writing – review & editing, Methodology, Funding acquisition, Conceptualization; **Liyuan Liu:** Writing – review & editing, Writing – original draft, Validation, Methodology; **Limin Pan:** Supervision, Investigation.

Data availability

The authors do not have permission to share data.

Declaration of competing interest

The authors declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

Acknowledgment

This research is partially supported by the 13th 5-year National Science and Technology Supporting Project, PR China under Grant 2018YFC2000302, the 2020 High Quality Development Project of Manufacturing Industry, PR China under Grant CEIEC-2020-ZM02-0134.

References

Almeida, A., Villers, J. P.D., Freitas, A.D., Velayudan, M., 2022. The complementarity of a diverse range of deep learning features extracted from video content for video recommendation. *Expert Syst. Appl.* 192, 116335. <https://doi.org/10.1016/j.eswa.2021.116335>

- Badjie, B., Cecilio, J., Casimiro, A., 2025. Adversarial attacks and countermeasures on image classification-based deep learning models in autonomous driving systems: a systematic review. *ACM Comput. Surv.* 57, 52. <https://doi.org/10.1145/3691625>
- Chen, H., He, X., Qing, L., Wu, Y., Ren, C., Sheriff, R.E., Zhu, C., 2022. Real-world single image super-resolution: a brief review. *Inf. Fus.* 79, 124–145. <https://doi.org/10.1016/j.inffus.2021.09.005>
- Chen, J., Shen, Y.Q., Jiang, J., Li, S., 2023. Toward understanding deep learning framework bugs. *ACM Trans. Softw. Eng. Methodol.* 32, 1–31. <https://doi.org/10.1145/3587155>
- Chib, P.S., Singh, P., 2024. Recent advancements in end-to-end autonomous driving using deep learning: a survey. *IEEE Trans. Intell. Veh.* 9, 103–118. <https://doi.org/10.1109/TIV.2023.3318070>
- Christou, N., Jin, D., Atlidakis, V., Ray, B., Kemerlis, V.P., 2023. IvySyn: automated vulnerability discovery in deep learning frameworks. In: *Proc 32th USENIX Conf. Secur. Symp., SEC, 2023*, pp. 7393–7410. <https://www.usenix.org/conference/usenixsecurity23/presentation/christou>
- Cui, Y., 2022. Deep learning for image and point cloud fusion in autonomous driving: a review. *IEEE Trans. Intell. Transp. Syst.* 23, 722–739. <https://doi.org/10.1109/TITS.2020.3023541>
- Deng, Y., Xia, C.S., Peng, H., Yang, C., Zhang, L., 2023. Large language models are zero-shot fuzzers fuzzing deep learning libraries via large language models, in: 32th. pp. 423–435. <https://doi.org/10.1145/3597926.3598067>
- Deng, Y., Xia, C.S., Yang, C., Zhang, S.D., Yang, S., Zhang, L., 2024. Large language models are edge-case generators: crafting unusual programs for fuzzing deep learning libraries. pp. 1–13. <https://doi.org/10.1145/3597503.3623343>
- Deng, Y., Yang, C., Wei, A., Zhang, L., 2022. Fuzzing deep-learning libraries via automated relational api inference, in: 30th. In: *ESEC/FSE*, pp. 44–56. <https://doi.org/10.1145/3540250.3549085>
- Deng, Y., Zhang, T., Lou, G., Zheng, X., Jin, J., Han, Q.L., 2021. Deep learning-based autonomous driving systems: a survey of attacks and defenses. *IEEE Trans. Ind. Inf.* 17, 7897–7912. <https://doi.org/10.1109/TII.2021.3071405>
- Deng, Z., Meng, G., Chen, K., Liu, T., Xiang, L., Chen, C., 2022. Differential testing of cross deep learning framework apis: revealing inconsistencies and vulnerabilities. In: *Proc 32th USENIX Conf. Secur. Symp., SEC, 2023*, pp. 7393–7410. <https://www.usenix.org/conference/usenixsecurity23/presentation/deng-zizhuang>
- Feldt, R., Magazini, A., 2010. Validity threats in empirical software engineering research - an initial survey. pp. 374–379. https://www.cse.chalmers.se/feldt/publications/feldt_2010_validity_threats_in_ese_initial_survey.pdf
- Go, G., Zhou, C., Zhang, Q., Zou, X., Shi, H., Jiang, Y., 2024. Towards more complete constraints for deep learning library testing via complementary set guided refinement. In: *Proc. 33rd*, pp. 1338–1350. <https://doi.org/10.1145/3650212.3680364>
- Gu, J.Z., Luo, X.C., Zhou, Y.F., Wang, X., 2022. Muffin: testing deep learning libraries via neural architecture fuzzing. In: *Proc ACM/IEEE 44th Int. Conf. Softw. Eng., ICSE*, pp. 1418–1430. <https://doi.org/10.1145/3510003.3510092>
- Guo, Q.Y. et al., 2020. Audex: automated testing for deep learning frameworks. *ASE*, pp. 486–498. <https://doi.org/10.1145/3324884.3416571>
- Harzevili, N.S., Wei, M., Mohajer, M.M., Pham, V.H., Wang, S., . Evaluating api-level deep learning fuzzers: A comprehensive benchmarking study. <https://doi.org/10.1145/3729533>
- Hatcher, W.G., Yu, W., 2018. A survey of deep learning: platforms, applications and emerging research trends. *IEEE Access* 6, 24411–24432. <https://doi.org/10.1109/ACCESS.2018.2830661>
- Islam, M.J., Nguyen, G., Pan, R., Rajah, H., 2019. Th. In: *A Comprehensive Study on Deep Learning Bug Characteristics*, pp. 510–520. *ESEC/FSE*. <https://doi.org/10.1145/3338906.3338955>
- Ji, P., Feng, Y., Wu, D., Yan, L., Chen, P., Liu, J., Zhao, Z., 2025. MoCo: fuzzing deep learning libraries via assembling code. *IEEE Trans. Softw. Engin* 51, 1939–3520. <https://doi.org/10.1109/TSE.2024.3509975>
- Li, M., Cao, J., Tian, Y., Li, T.O., Wen, M., Cheung, S.C., 2023a. COMET: coverage-guided model generation for deep learning library testing. *ACM Trans. Softw. Eng. Methodol.* 32, 1–34. <https://doi.org/10.1145/3583566>
- Li, X., Yang, X., Ma, Z., Xue, J., 2023b. Deep metric learning for few-shot image classification: a review of recent developments. *Pattern Recognit* 138, 109381. <https://doi.org/10.1016/j.patcog.2023.109381>
- Liu, J., Huang, Y., Wang, Z., Ma, L., Fang, C., Gu, M., Zhang, X., Chen, Z., 2023. Generation-based differential fuzzing for deep learning libraries. *ACM Trans. Softw. Eng. Methodol.* 33, 1–28. <https://doi.org/10.1145/3628159>
- Luo, W., Chai, D., Run, X., Wang, J., Fang, C., Chen, Z., 2021. Graph-based fuzz testing for deep learning inference engines. In: *Proc. 43rd Int. Conf. Softw. Eng., ICSE, 2021*, pp. 288–299. <https://doi.org/10.1109/ICSE43902.2021.00037>
- Lyu, Y., Xie, Y., Chen, P., Chen, H., 2024. Prompt fuzzing for fuzz driver generation. In: *Proc ACM SIGSAC Conf. Comp. Commun. Secur., CCS, 2024*, pp. 3793–3807. <https://doi.org/10.1145/3658644.3670396>
- Nie, Y., Xiao, X., Yang, B., Li, H., Luo, L., Yu, H., Sun, G., 2024. Python coverage guided fuzzing for deep learning framework. In: *Int. Conf. Electr. Eng.* pp. 1–6. <https://doi.org/10.1109/EEIS62553.2024.00007>
- Pham, H.V., Lutellier, T., Qi, W., Tan, L., 2019. CRADLE: cross-backend validation to detect and localize bugs in deep learning libraries, in: 41th. pp. 1027–1038. *ICSE*. <https://doi.org/10.1109/ICSE.2019.00107>
- Shi, J., Xiao, Y., Li, Y., Li, Y., Yu, D., Yu, C., Su, H., Chen, Y., Huo, W., 2023. ACETest: automated constraint extraction for testing deep learning operators, in: 32th. pp. 690–702. <https://doi.org/10.1145/3597926.3598088>
- Tao, Z., Liu, X., Xia, Y., Wang, X., Yang, L., Huang, X., Chua, T.S., 2023. Self-supervised learning for multimedia recommendation. *IEEE Trans. Multimed.* 25, 5107. <https://doi.org/10.1109/TMM.2022.3187556>

- Touvron, H., 2023. ResMLP: feedforward networks for image classification with data-efficient training. *IEEE Trans. Pattern Anal. Mach. Intell.* 45, 1939–3539. <https://doi.org/10.1109/TPAMI.2022.3206148>
- Wan, J., Lutellier, T., Qian, S., Phan, H.V., Tan, L., 2022. Eagle: creating equivalent graphs to test deep learning libraries. In: *ACM/IEEE 44th Int. Conf. Softw. Eng., ICSE*, pp. 798–810. <https://doi.org/10.1145/3510003.3510165>
- Wang, Q., Wei, Y., Yin, J., Wu, J., Song, X., Nie, L., 2023. DualGNN: dual graph neural network for multimedia recommendation. *IEEE Trans. Multimed.* 25, 1074–1084. <https://doi.org/10.1109/TMM.2021.3138298>
- Wei, A., Deng, Y., Yang, C., Zhang, L., 2022. Free lunch for testing: fuzzing deep-learning libraries from open source. pp. 995–1007. *ICSE*. <https://doi.org/10.1145/3510003.3510041>
- Wei, A.J., Deng, Y.L., Yang, C.Y., Zhang, L.M., 2022. Free lunch for testing: fuzzing deep-learning libraries from open source. In: *ACM/IEEE 44th Int., Conf. Softw. Eng., ICSE*, 2022, pp. 995–1007. <https://doi.org/10.1145/3510003.35100>
- Wu, L., He, X., Wang, X., Zhang, K., Wang, M., 2023. A survey on accuracy-oriented neural recommendation: from collaborative filtering to information-rich recommendation. *IEEE Trans. Knowl. Data Eng.* 35, 4425–4445. <https://doi.org/10.1109/TKDE.2022.3145690>
- Xie, D., Li, Y., Kim, M., Pham, H.V., Tan, L., Zhang, X., Godfrey, M.W., 2022. Docter: documentation-guided fuzzing for testing deep learning api functions, in: 31th. pp. 176–188. <https://doi.org/10.1145/3533767.3534220>
- Yang, C., Deng, Y., Yao, J., Tu, Y., Li, H., Zhang, L., 2023. Fuzzing automatic differentiation in deep-learning libraries. pp. 1174–1186. <https://doi.org/10.1109/ICSE48619.2023.00105>
- Yuan, Y., Wang, C., Jiang, Z., 2020. Proxy-based deep learning framework for spectral-spatial hyperspectral image classification: efficient and robust. *IEEE Trans. Geosci. Remote Sens.* 60, 1–15. <https://doi.org/10.1109/TGRS.2021.3054008>
- Zhang, X., et al., 2021a. Duo: differential fuzzing for deep learning operators. *IEEE Trans. Reliab.* 70, 1671–1685. <https://doi.org/10.1109/TR.2021.3107165>
- Zhang, X., et al., 2021b. Predoo: precision testing of deep learning operators, in: 30th. pp. 400–412. <https://doi.org/10.1145/3460319.3464843>
- Zhang, X., Jiang, W., Li, Q., Wang, Q., Lin, C., Guan, X., 2025. Deep learning library testing: definition, methods and challenges. *ACM Comput. Surv.* 57, 1–37. <https://doi.org/10.1145/3716497>